

CS780 Capstone Project: OBELIX: the Warehouse Robot

Ritika
230862
{ritika23}@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This work studies learning control policies for the OBELIX robot to locate, push, and unwedge a box in a constrained environment. A range of deep reinforcement learning methods were evaluated, including Double/Dueling DQN, D3QN with prioritized experience replay, and PPO variants with and without LSTM. Comparative experiments analyze value-based vs policy-gradient approaches, along with trajectory-guided signals and performance across difficulty settings. Results highlight challenges in exploration and recovery, and demonstrate improved robustness in handling stuck states and coordinated box manipulation.

1 Competition Result

Codabench Username: r.230862

Final leaderboard rank (Test Phase): 60

Leaderboard rank – Level 1 (static): 53

Leaderboard rank – Level 2 (blinking): 29

Leaderboard rank – Level 3 (moving + blinking): 32

Number of valid submissions made (Development(Level 1 , Level 2, Level 3) and Test Test):

Category	Number of Submissions
Development (Level 1)	6
Development (Level 2)	5
Development (Level 3)	2
Test	2

Table 1: Number of Valid Submissions

2 Introduction and Problem Description

The objective of this project is to learn control policies for the OBELIX robot to execute a structured manipulation pipeline: **find** $\rightarrow$ **attach** $\rightarrow$ **push** $\rightarrow$ **unwedge** a grey box within a bounded 2D arena. Unlike standard navigation tasks, successful completion requires not only reaching the object but also maintaining sustained physical interaction to push it to the boundary while handling collisions and deadlocks.

The environment is a continuous 2D workspace discretized through rendering. The robot is modeled as a circular agent with discrete actions: rotations ($\pm 22.5^\circ$, $\pm 45^\circ$) and forward motion. Perception is limited to **8 directional sonar sectors** (with near and far range encoding) and a **forward-facing**

infrared sensor, resulting in an 18-dimensional binary observation vector. These sensors detect the presence of objects (box or obstacles) within angular sectors, without providing explicit position or velocity information.

The grey box is a rigid body that becomes interactable only after **physical contact (attachment)**, detected via overlap between the robot and box masks. Once attached, the robot can push the box; however, motion is constrained by collision checks with arena boundaries and optional wall obstacles. Failed forward motion results in a **stuck condition**, requiring the agent to reorient and recover (unwedge).

The environment supports multiple difficulty modes:

- **Static (difficulty 0):** box remains stationary and fully observable.
- **Blinking (difficulty ≥ 2):** box visibility toggles stochastically, breaking continuous perception.
- **Moving + Blinking (difficulty ≥ 3):** box follows a random velocity with occasional direction changes, while also blinking, introducing strong non-stationarity.

Additionally, **wall obstacles** (with narrow gaps) create constrained pathways and increase the likelihood of the robot-box system getting trapped during pushing.

The reward structure is sparse and interaction-driven: a large bonus for successfully pushing the box to the boundary, a one-time reward for attachment, small per-step penalties, and heavy penalties for stuck states. Importantly, rewards are tied to **sensor activations and contact events**, rather than explicit distance-based shaping.

This problem is inherently a **Partially Observable Markov Decision Process (POMDP)**, since the agent must infer the state of the box (position, visibility, motion) from local, noisy, and temporally sparse observations. The key challenges include partial observability, delayed rewards, contact-rich dynamics, and recovery from failure states.

The overall goal is to learn policies that maximize **mean cumulative reward across difficulty settings**, while demonstrating robustness to dynamic targets, intermittent observability, and obstacle-induced deadlocks.

3 Approach and Model Evolution

3.1 Baseline / Initial Attempts

Initial exploration used value-based reinforcement learning methods including **DQN**, **Double DQN (DDQN)**, **Dueling DQN (D3QN)**, and **D3QN with Prioritized Experience Replay (PER)** on lower difficulty levels.

Outcome: All approaches failed to learn meaningful navigation and converged to a degenerate policy of continuous rotation.

Key Issues:

- Partial observability due to limited 18-dimensional state
- Sparse and delayed reward signals
- Action imbalance (multiple turning actions vs single forward action)
- Instability of off-policy learning in dynamic environments

3.2 Evolution Across Phases

Phase 1: PPO (MLP)

Switched to **Proximal Policy Optimization (PPO)** for improved stability. Training became more stable, but the learned policy remained rotation-dominated.

Phase 2: PPO + LSTM

An **LSTM-based actor-critic** was introduced to handle partial observability and capture temporal dependencies (e.g., blinking/moving box).

Phase 3: Curriculum Learning

Training was performed progressively:

- Difficulty 0: Static box
- Difficulty 2: Blinking box
- Difficulty 3: Moving box (with walls)

Weights from each stage were used to initialize the next.

Phase 4: Reward Shaping

Structured reward signals were introduced:

- Distance-based shaping
- Forward action bias
- Entropy regularization
- Reward normalization

3.3 Final Strategy

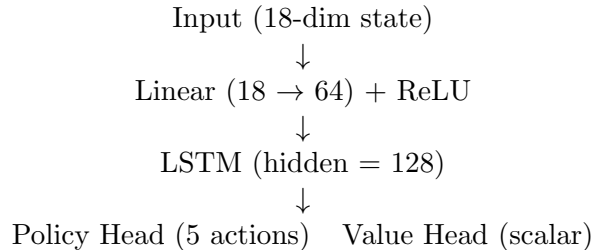
The final model uses **PPO with LSTM**, trained via curriculum learning and enhanced with reward shaping and entropy-based exploration.

3.4 Final Model Details

Algorithm:

- PPO (clipped objective)
- Actor-Critic with Generalized Advantage Estimation (GAE)
- $\gamma = 0.99$, $\lambda = 0.95$

Architecture:



State Representation:

- Raw 18-dimensional sensor input
- No handcrafted features

Exploration:

- Stochastic policy (Categorical distribution)
- Entropy bonus with slow decay (minimum = 0.01)

Reward Shaping:

- Approach reward: $+3\times$ (reduction in distance to box)
- Action bias: $+2$ for forward, -0.5 for turning
- Reward normalization using running mean and variance

Training Details:

- Backpropagation through time (chunk length = 64)
- Gradient clipping
- Entropy decay

3.5 Key Observations

- Convergence to rotation loops is frequently observed
- Forward movement remains inconsistent
- Reliable interaction with the box is not achieved

3.6 Core Challenges

- Weak directional signal in the state representation
- Strong bias toward turning actions
- Presence of local optima (rotation behavior)
- Difficult credit assignment due to delayed rewards

3.7 Summary

Stage	Method	Result
Initial	DQN / DDQN / D3QN / PER	Failed
Intermediate	PPO (MLP)	Stable but ineffective
Final	PPO + LSTM + Reward Shaping	Best attempt

4 Experiments and Implementation Details

4.1 Training Setup

- Episodes: up to 4000
- Max steps: 1500
- Hardware: AMD Ryzen 5 CPU (no GPU)

4.2 Optimization

- PPO (on-policy), Adam optimizer
- Learning rate: 2×10^{-4}
- $\gamma = 0.99$, $\lambda = 0.95$
- Clip: 0.2, Epochs: 8
- Gradient clipping applied

4.3 Training Details

- BPTT with chunk length = 64
- Entropy bonus: $0.05 \rightarrow 0.01$ (decay)
- Reward normalization (running mean/std)

4.4 Curriculum Learning

- Difficulty $0 \rightarrow 2 \rightarrow 3$
- Pretrained weights used across stages

4.5 Hyperparameters (Final Model)

Parameter	Value
Learning rate	2×10^{-4}
Discount factor (γ)	0.99
GAE (λ)	0.95
PPO clip	0.2
Epochs per update	8
Chunk length	64
Entropy coef (init)	0.05
Entropy coef (min)	0.01

4.6 Model Variants

Model	Outcome
DQN / DDQN / D3QN / PER	Failed
PPO (MLP)	Unstable policy
PPO + LSTM	Final model

4.7 Evaluation

- Best model selected via highest episode return
- Stochastic policy + entropy reduces overfitting

5 Results

5.1 Development Phase Performance

Level	Walls	Mean Reward	Std Dev
1	True	-1981.2	7.692
1	False	-1990.9	10.445
2	True	-1980.7	6.900
2	False	-1989.2	11.626
3	True	-1530.7	1173.348
3	False	-1491.3	1199.430

Weighted Reward (Level 3): -1514.94

5.2 Final Test Phase Performance

Difficulty	Walls	Mean Reward	Std Dev
0	True	-1981.2	7.692
2	True	-1981.2	7.692
3	True	-1530.9	1173.423
0	False	-1990.9	10.445
2	False	-1990.9	10.445
3	False	-1491.3	1199.430

Aggregate Results:

- With walls: Mean = -1831.10, Std = 709.98
 - Without walls: Mean = -1824.37, Std = 731.49
 - Weighted cumulative reward: -1828.41
-

5.3 Learning Behaviour

- No consistent upward trend in returns across training
 - High variance observed in Level 3 (dynamic environment)
 - Agent often converges to repetitive rotation policies
-

5.4 Best Model Analysis

- Best performance observed with **PPO + LSTM** model
- Slight improvement in Level 3 compared to lower levels

Reasoning:

- LSTM provides limited temporal context (helps in dynamic settings)

- PPO offers more stable updates compared to value-based methods

Limitations:

- Poor generalization across levels
- Low sample efficiency due to weak reward signal
- Lack of robustness (high variance in results)

6 Error Analysis and Discussion

6.1 Failure Modes by Difficulty Level

Level 1 (Static Box):

- Occasional failure due to lack of clear directional encoding in the 18-bit observation.
- Agent sometimes converges to local rotation loops instead of approaching the box.
- Sparse reward signal makes early learning unstable.

Level 2 (Blinking Box):

- Agent loses track of the box during invisible phases (partial observability).
- Leads to inconsistent trajectories and frequent resets to exploratory behavior.
- LSTM helps partially, but not sufficient for reliable tracking.

Level 3 (Moving Box):

- Difficulty in predicting motion and intercepting the box.
- Policy reacts myopically instead of learning anticipatory behavior.
- High variance in returns indicates unstable learning.

With Walls:

- Frequent collisions and poor navigation around obstacles.
- No explicit spatial reasoning or obstacle avoidance mechanism.
- Agent often gets stuck in corners or oscillatory movement patterns.

6.2 Limitations

- **Weak state representation:** 18-bit input does not provide sufficient spatial or directional information.
- **Action imbalance:** Multiple turning actions bias policy toward rotation.
- **Reward sparsity:** Delayed success signal makes credit assignment difficult.
- **Limited planning:** LSTM captures short-term memory but not long-term trajectory planning.

6.3 Potential Improvements

- Incorporate richer state features (e.g., relative position or velocity estimates).
- Redesign action space to reduce rotational bias.
- Use auxiliary tasks (e.g., direction prediction) to guide learning.
- Introduce hierarchical or behavior-based policies (explore vs approach vs push).
- Improve reward shaping with more informative intermediate signals.

6.4 Insights

- Rotation is an easy-to-learn but suboptimal local optimum.
- Memory alone (LSTM) does not solve partial observability without informative inputs.
- Reward shaping significantly affects behavior, but cannot fully compensate for weak observations.
- Curriculum learning improves stability but does not guarantee transfer to higher difficulty.

7 Conclusion

Key Achievements: Multiple reinforcement learning paradigms were explored, including value-based methods (DQN variants) and policy-gradient approaches (PPO). An LSTM-based PPO actor-critic model was designed to address partial observability, along with curriculum learning and reward shaping to improve training stability across increasing difficulty levels.

Main Takeaway: In partially observable robotic tasks, performance is fundamentally constrained by the quality of the state representation. The limited observation space fails to encode sufficient directional or spatial information, making it difficult for the agent to learn robust navigation strategies. Even with advanced methods such as PPO combined with LSTM and entropy-based exploration, learning remains affected by sparse rewards, action imbalance, and convergence to suboptimal behaviors such as rotation.

Future Directions: Future improvements should focus on enhancing the observation space through richer features or learned representations, and reducing bias in the action space. Stronger POMDP-oriented approaches, such as attention mechanisms or belief state estimation, may improve decision-making under uncertainty. Additionally, hierarchical or behavior-based policies and extensions to more realistic environments, including multiple objects and complex dynamics, could further improve performance and generalization.

Code Repository: [GitHub Link](#) (Includes training scripts, trained model weights, and the final agent implementation.)

Video Demonstration

A demonstration comparing PPO with frame stacking and PPO with LSTM is available here:
[Watch Video](#)

LLM Usage Declaration

- Which Large Language Model (LLM) did you use to complete the competition or write the report?
Chatgpt
- In which part did you use the LLM, and how did it help you?

Part	Used LLM? (Yes/No)	Purpose of Using LLM
Understanding concepts	Yes	Clarifying PPO, LSTM, and RL training details
Ideation / brainstorming	Yes	Discussing possible approaches and improvements
Debugging small code snippets	Yes	Fixing errors (e.g., dtype issues, training bugs)
Plotting / visualization code	Yes	understanding the wroking of visualization code
Grammar / writing style	Yes	Refining report wording and formatting
Report structure / section ideas	Yes	Organizing sections and formatting in LaTeX
Other (specify)	No	

- Did you refer to any other sources or websites apart from the LLM and lecture slides?
Yes. Standard references such as Sutton & Barto (Reinforcement Learning book) and the OBELIX GitHub repository were referred.
- Apart from any LLM usage, what was your own contribution?
The problem formulation, implementation of all models, debugging of training issues, reward design, and experimentation across difficulty levels were done independently. This includes designing the PPO + LSTM architecture, implementing training with BPTT, tuning hyperparameters, and analyzing failure cases across levels. The final report content and conclusions are based on personal experimentation and observations.

References

- [1] H. Van Hasselt, A. Guez, and D. Silver, *Deep Reinforcement Learning with Double Q-learning*, Proceedings of the AAAI Conference on Artificial Intelligence, vol. 30, no. 1, 2016.
- [2] S. Mahadevan and J. Connell, *Automatic programming of behavior-based robots using reinforcement learning*, Proceedings of AAAI-91, 1991.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.
- [4] Exploration Lab, IIT Kanpur, *CS780 OBELIX Environment and Codebase*, 2024. Available: <https://github.com/Exploration-Lab/CS780-OBELIX> (Accessed: 2026).